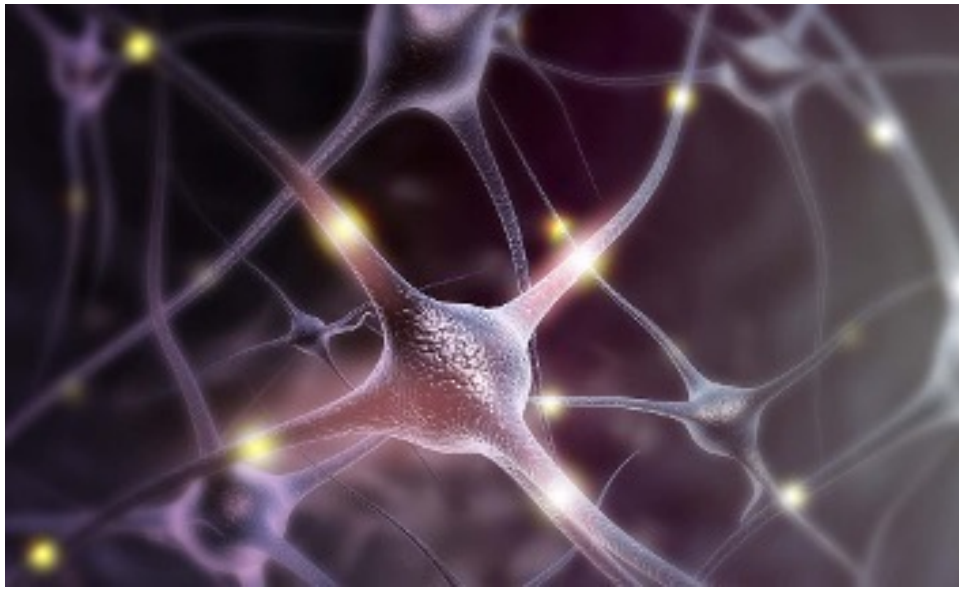
ASTR4004/8004 Computational Astronomy

Neural Networks

Yuxiang.Qin@anu.edu.au
Duffield Building D.115



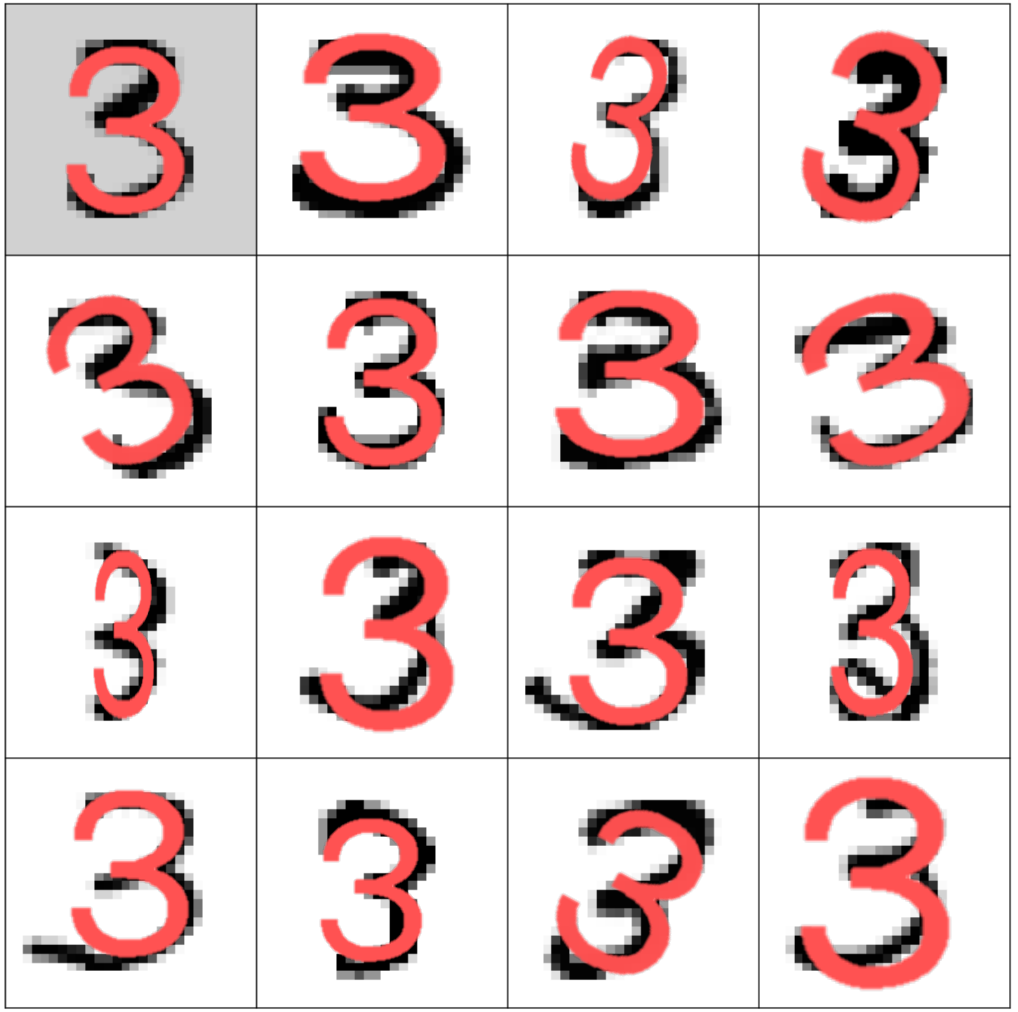
Australian National University

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY)
CRICOS PROVIDER CODE: 00120C

Quiz time

What are these?

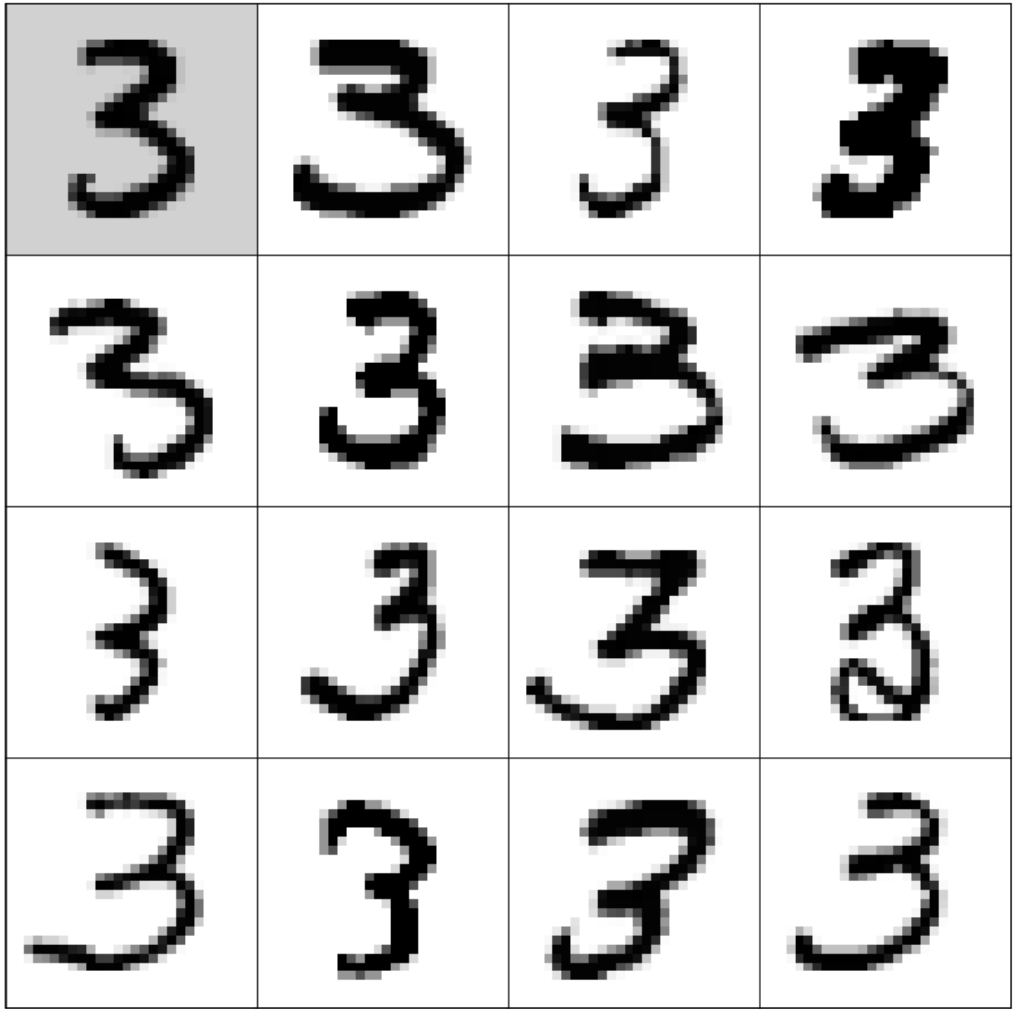
3



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Quiz time

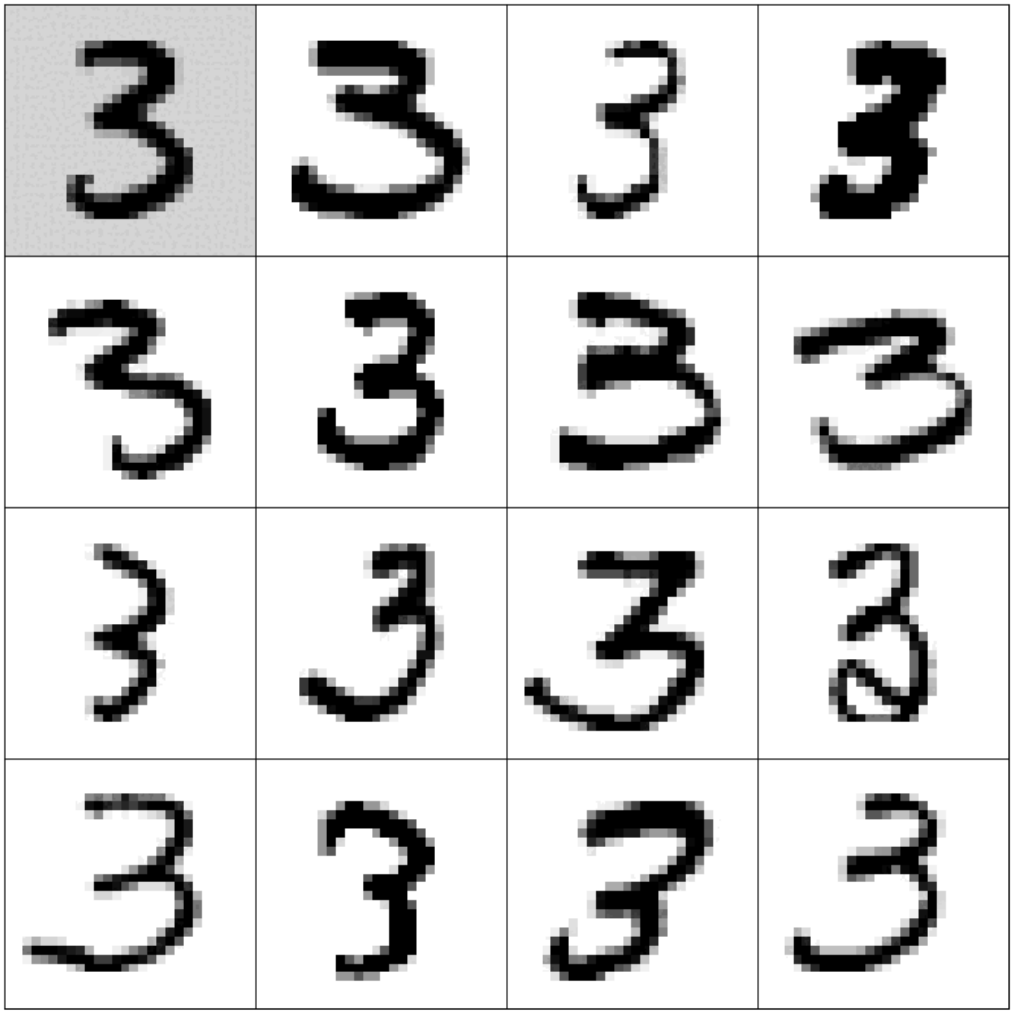
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0.5	0.9	1	1	0.7	0	0	0	0	0
0	0	0	0	0.4	0.3	0.4	0.5	1	0.5	0	0	0	0
0	0	0	0	0	0	0	0.6	1	0.5	0	0	0	0
0	0	0	0	0	0.3	0.9	0.9	0.5	0	0	0	0	0
0	0	0	0	0	1	1	0.9	0.4	0	0	0	0	0
0	0	0	0	0	0.1	0.1	0.5	0.9	0.6	0	0	0	0
0	0	0	0	0	0	0	0	0.2	1	0.4	0	0	0
0	0	0	0.1	0.4	0	0	0	0.4	1	0.4	0	0	0
0	0	0	0.4	0.7	0.3	0.5	0.8	1	0.7	0	0	0	0
0	0	0	0	0.6	1	1	0.8	0.4	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

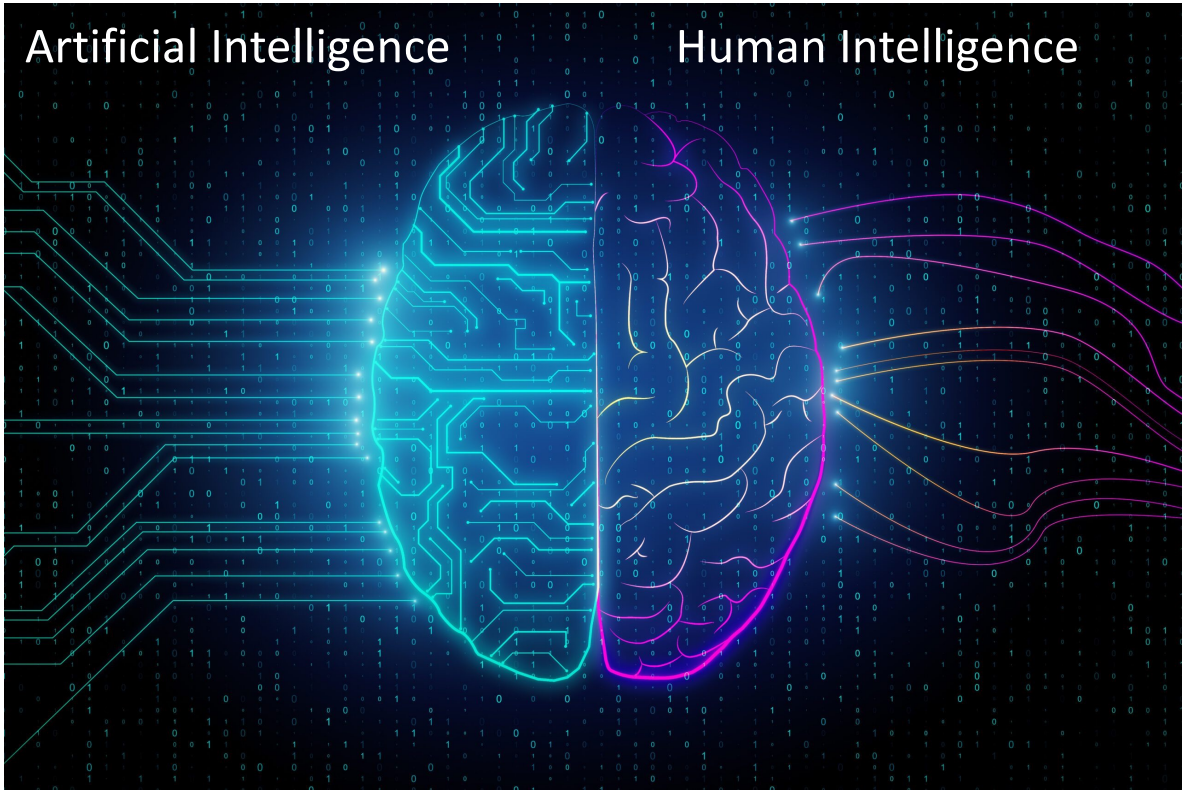
Quiz time

0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0.5	0.9	1	1	0.7	0	0	0	0	0
0	0	0	0	0.4	0.3	0.4	0.5	1	0.5	0	0	0	0
0	0	0	0	0	0	0	0.6	1	0.5	0	0	0	0
0	0	0	0	0	0.3	0.9	0.9	0.5	0	0	0	0	0
0	0	0	0	0	1	1	0.9	0.4	0	0	0	0	0
0	0	0	0	0	0.1	0.1	0.5	0.9	0.6	0	0	0	0
0	0	0	0	0	0	0	0	0.2	1	0.4	0	0	0
0	0	0	0.1	0.4	0	0	0	0.4	1	0.4	0	0	0
0	0	0	0.4	0.7	0.3	0.5	0.8	1	0.7	0	0	0	0
0	0	0	0	0.6	1	1	0.8	0.4	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

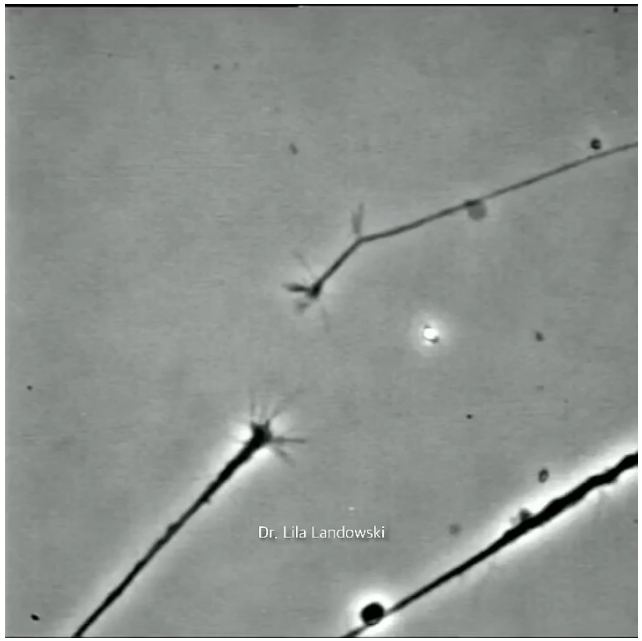
Neural Network



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

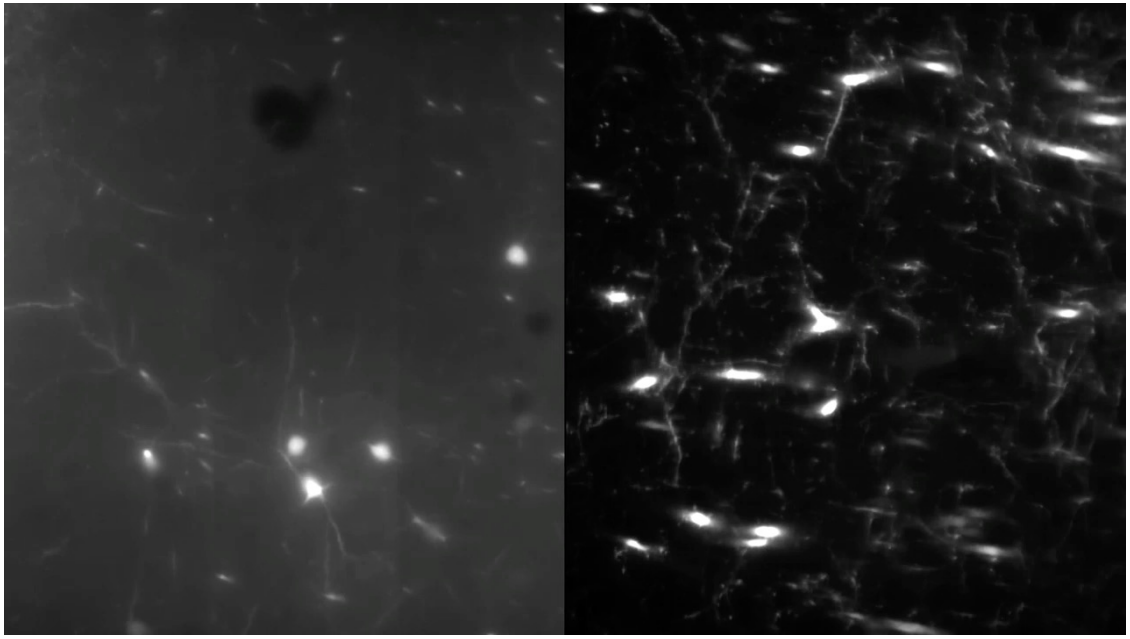
Biological Neural Network

Two neurons connecting to each other



Credit: Lila Landowski

Large-scale 3D tissue structure



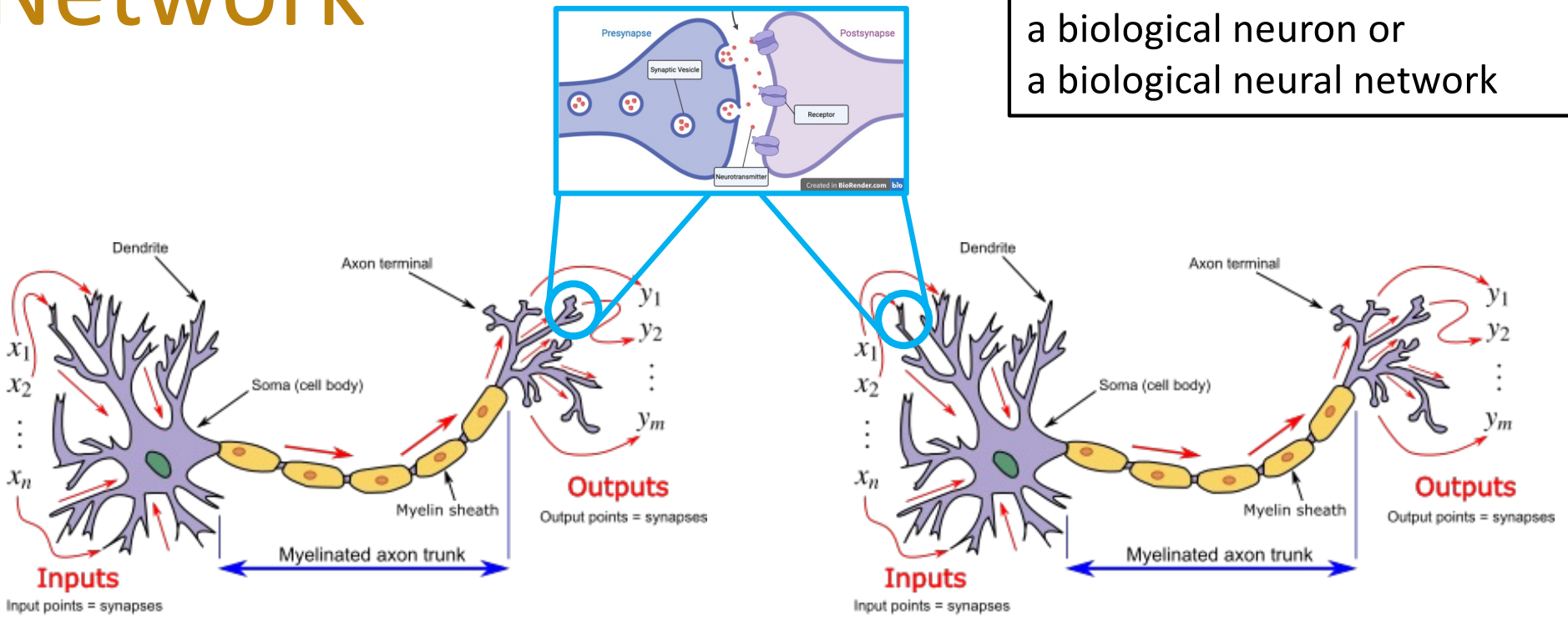
Credit: Yanlu Chen

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Biological Neural Network

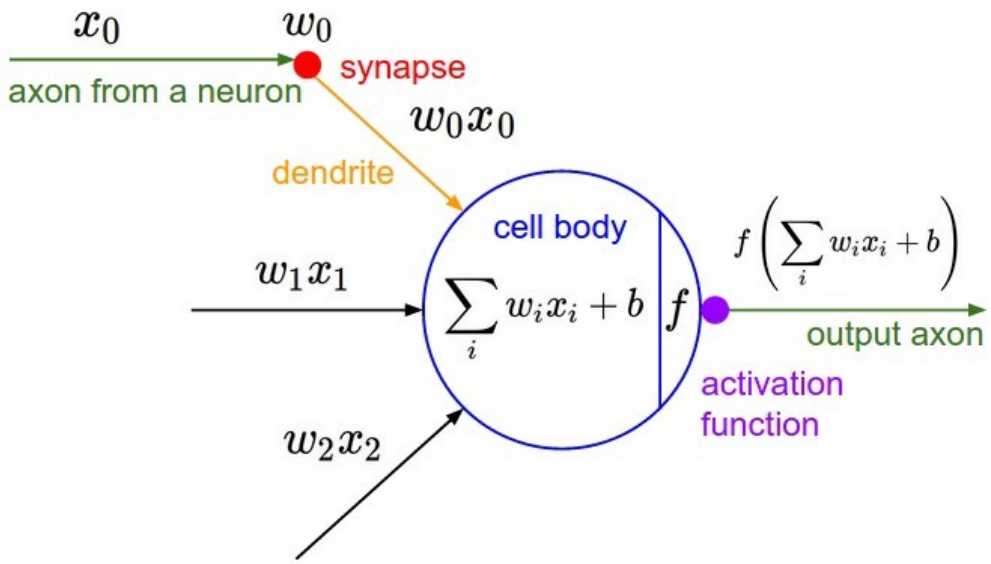
???

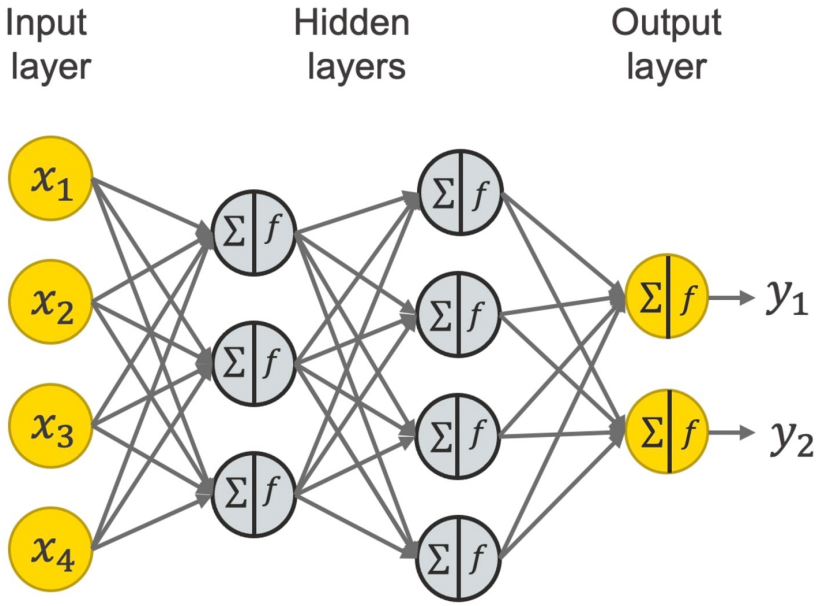
What is the simplest mathematical model for a biological neuron or a biological neural network



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Artificial Neural Network





TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

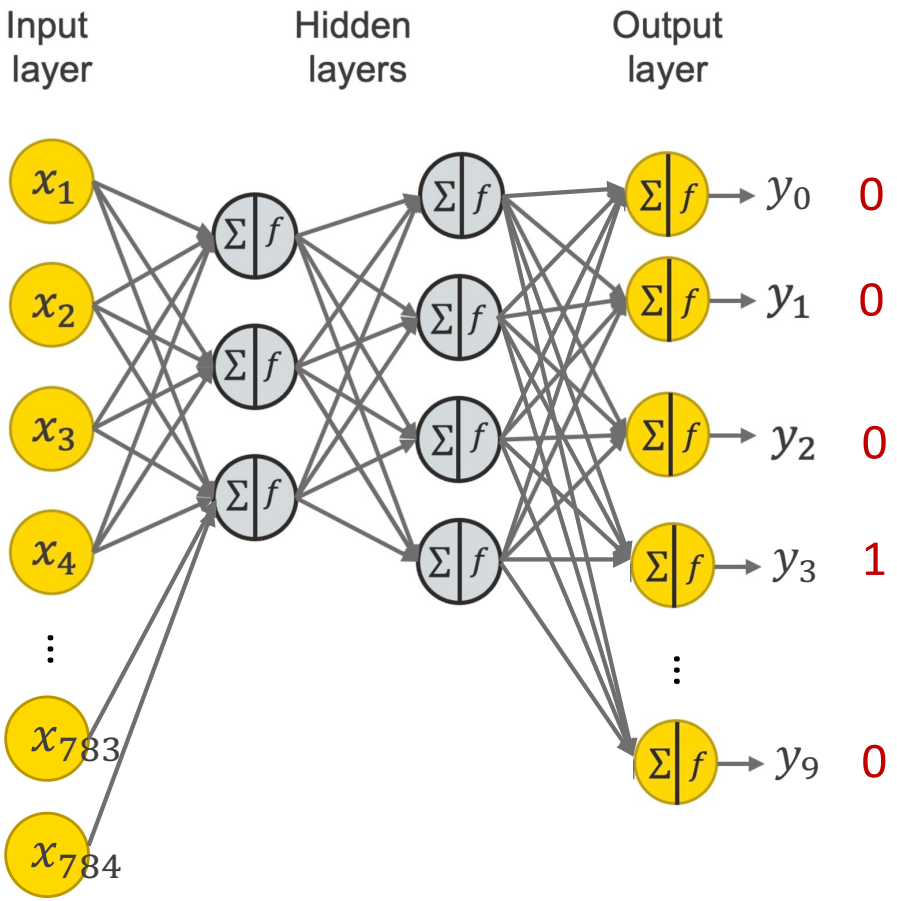
Artificial Neural Network



INPUT
(28x28)

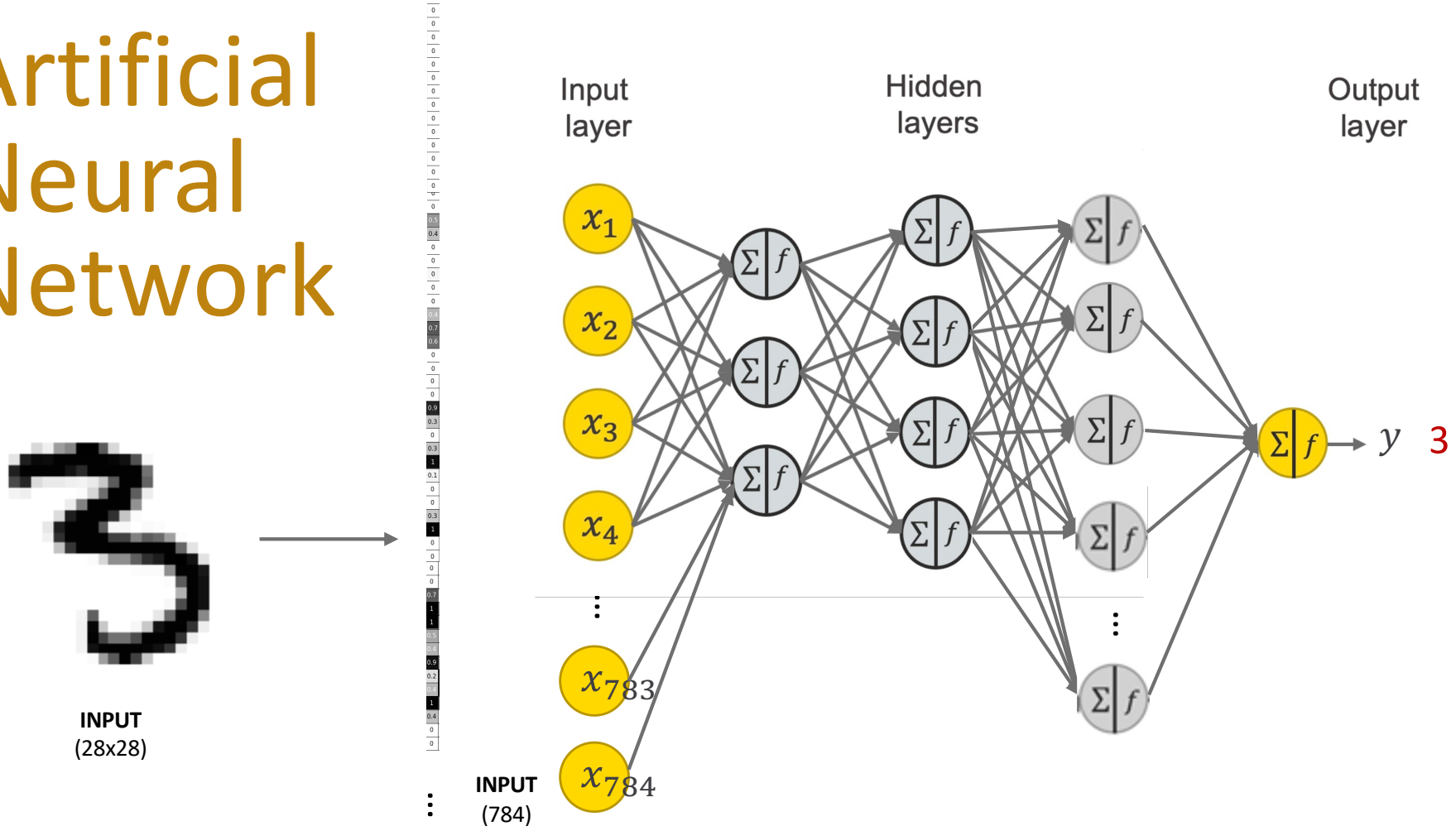


INPUT
(784)



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

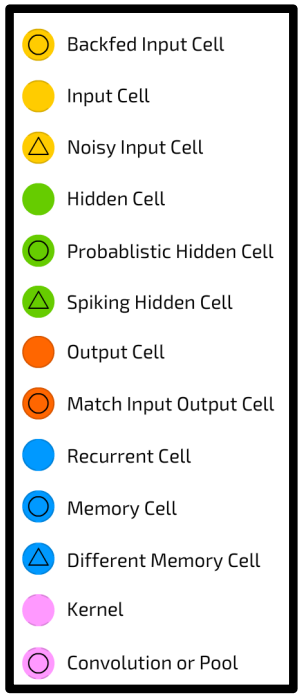
Artificial Neural Network

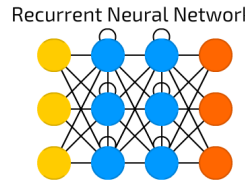


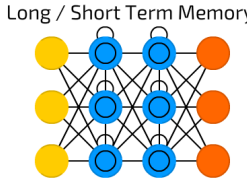
TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

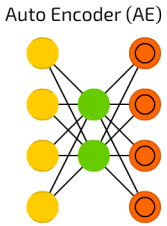
Artificial Neural Network

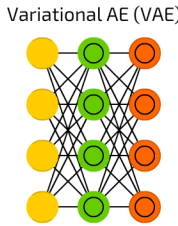
Things can get (very) interesting from here

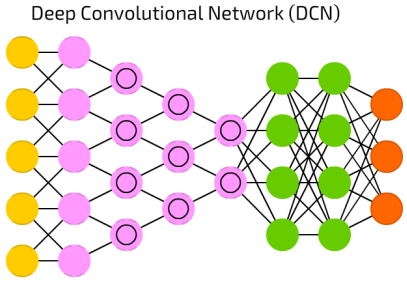


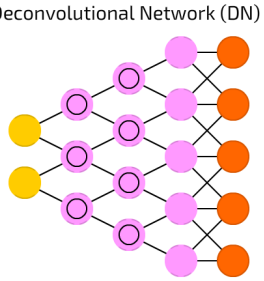


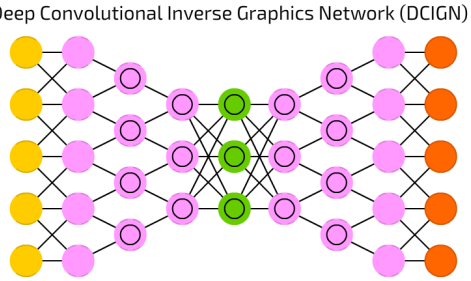


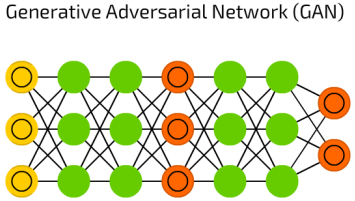


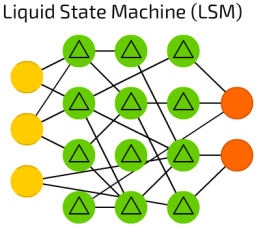


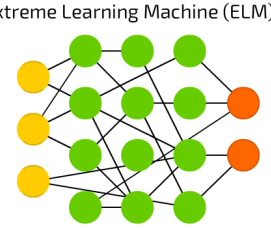


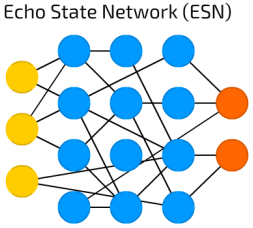


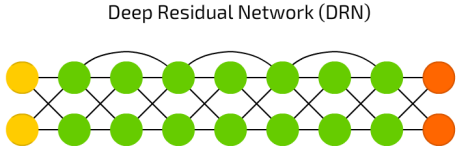


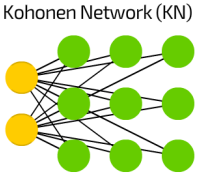


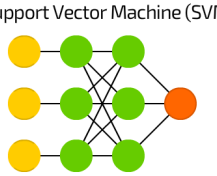


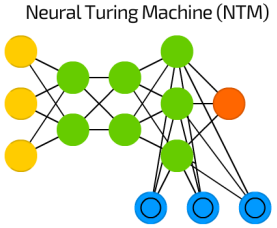












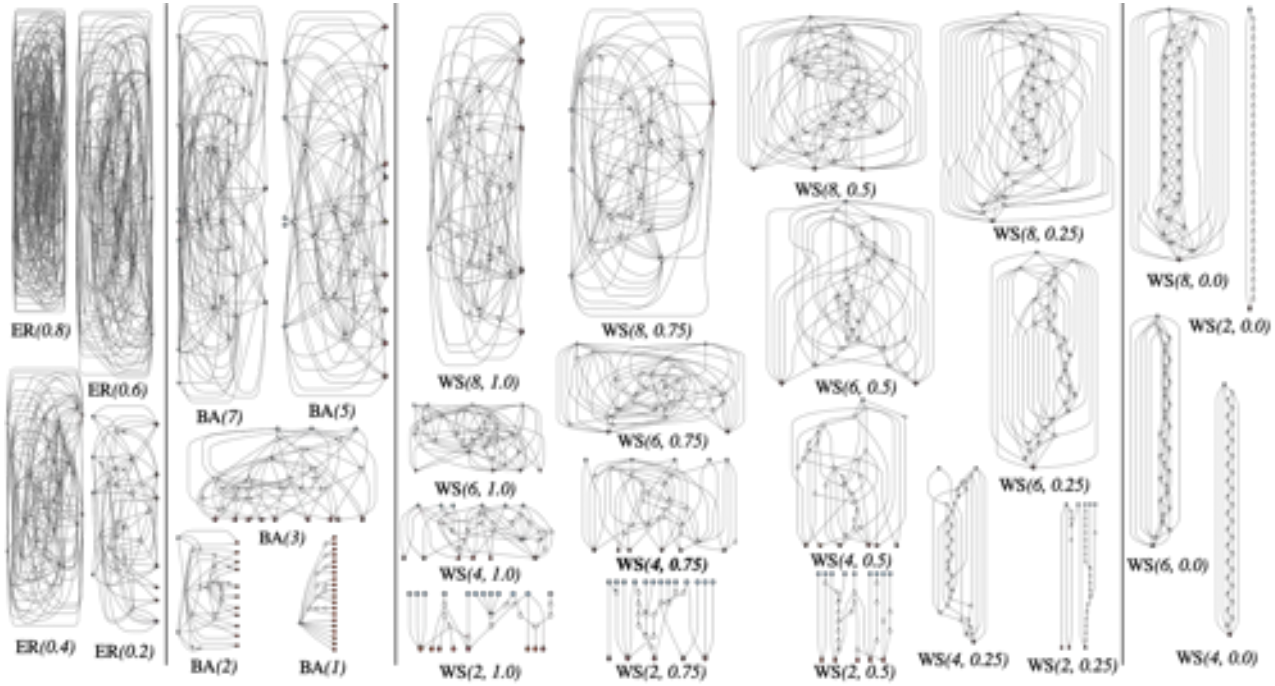
TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Artificial Neural Network

Things can get (very) interesting from here

"the physical connections of the nervous system ... are not identical from one organism to another", and "at birth, the construction of the most important networks is largely random."

-- Frank Rosenblatt (1958)



Xie+19

ER(P) [Erdos-Renyi] two nodes are connected with a probability of P

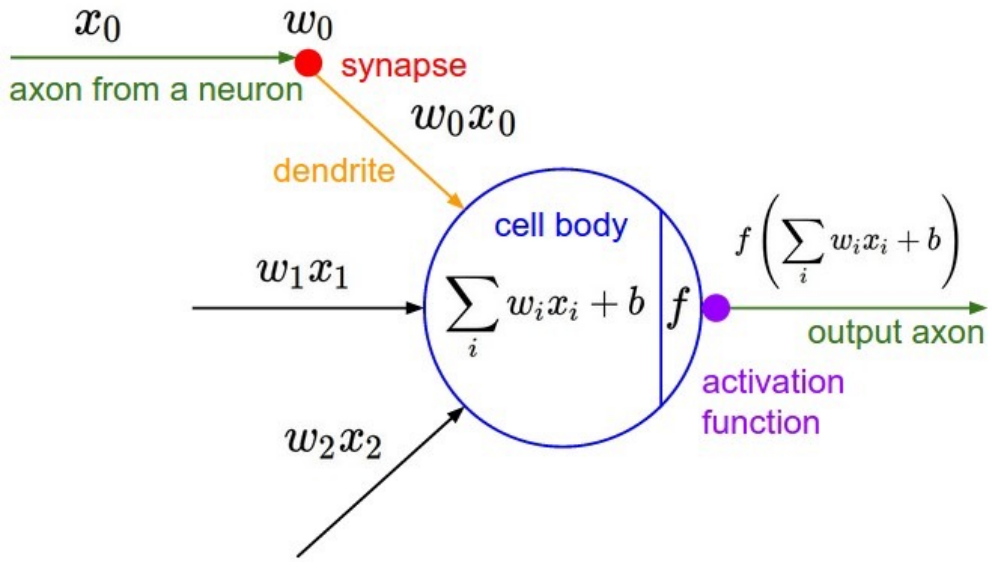
BA(M) [Barabasi-Albert], initial state has M nodes with no connection, new nodes are added and connected to existing node with a probability proportional to its degree.

WS(K,P) [Watts-Strogatz], initial state has N nodes regularly placed in a circle, and each connected to its K/2 neighbours on both side, every node is then connected to the ith next node with a probability of P

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Neurons

- Receives inputs (an N-D array)
- Multiples each input by its weight (w; coefficient)
- Applies activation function (f) to the sum and bias (b)
- Output result



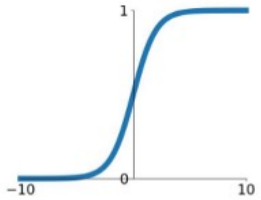
TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Activation functions

- Controls when the unit is active or inactive.
- Adds non-linearity to the neural network.

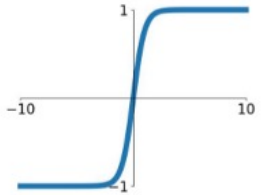
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



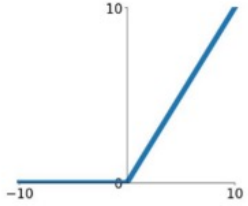
tanh

$$\tanh(x) = \frac{e^{x}-e^{-x}}{e^{x}+e^{-x}}$$



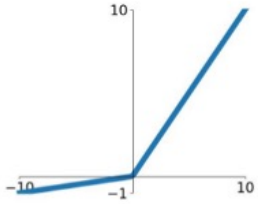
ReLU

$$\max(0, x)$$



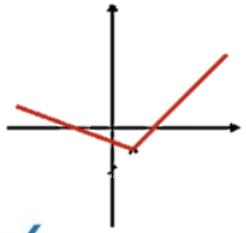
Leaky ReLU

$$\max(0.1x, x)$$



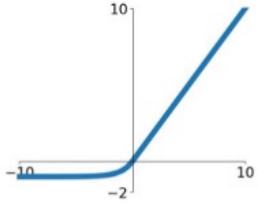
Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$



ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^{x} - 1) & x < 0 \end{cases}$$



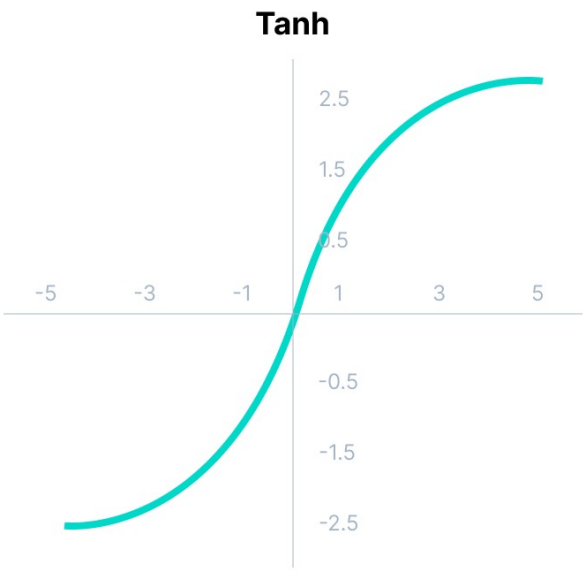
TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Activation functions

There is no good or bad AF. Things to consider:

- Zero center or not, e.g., the output of the tanh activation function is zero centred; hence we can easily map the output values as strongly negative, neutral, or strongly positive.



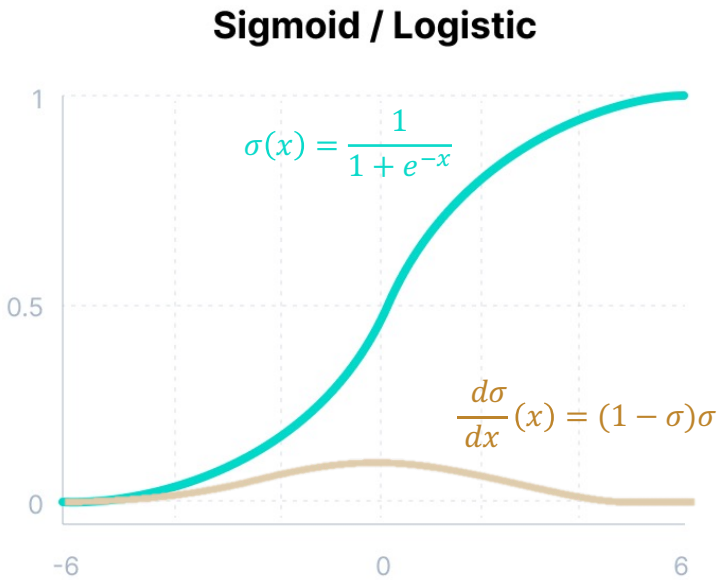


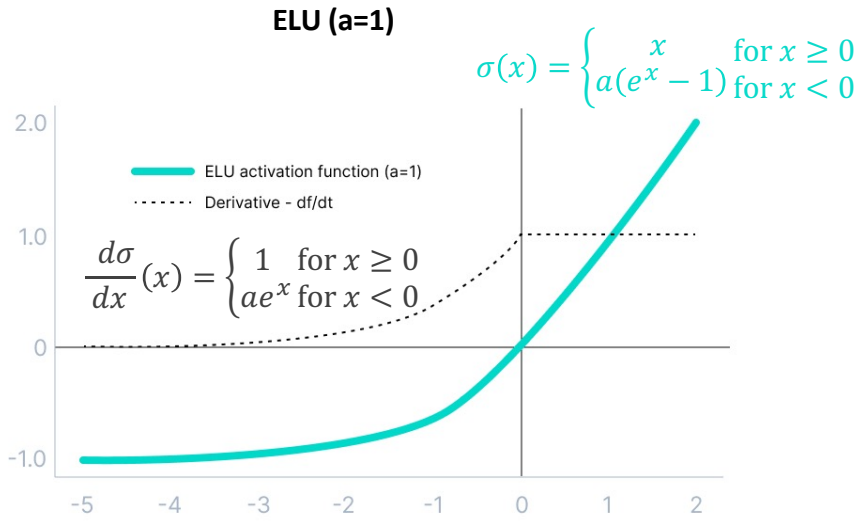
TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Activation functions

There is no good or bad AF. Things to consider:

- Gradient! You would like it to vanish (in most cases)
  - as the gradient value approaches zero, the network ceases to learn.
  - gradient can also accumulate and result in very large updates to the weight, causing overflow and hence unstable network.



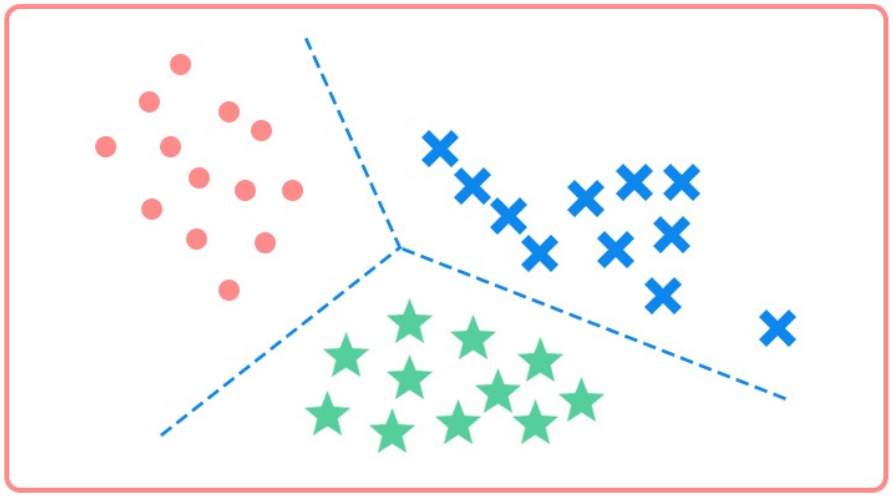


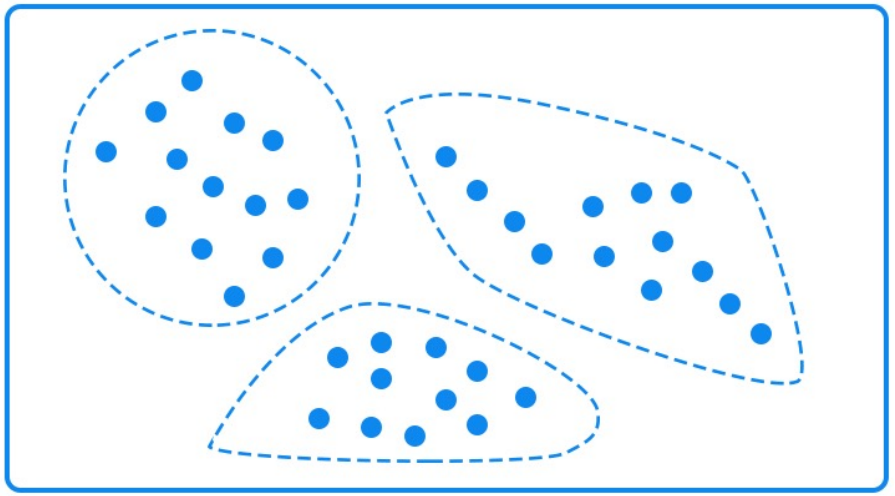
TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Supervised vs unsupervised

It's about the task you are interested

- Pre-categorized data or unlabeled data
- Interested in predictions or patterns
- Classification (divide the socks by color) or clustering (divide by similarity)
- Regression (divide the ties by length) or association (identify sequences)

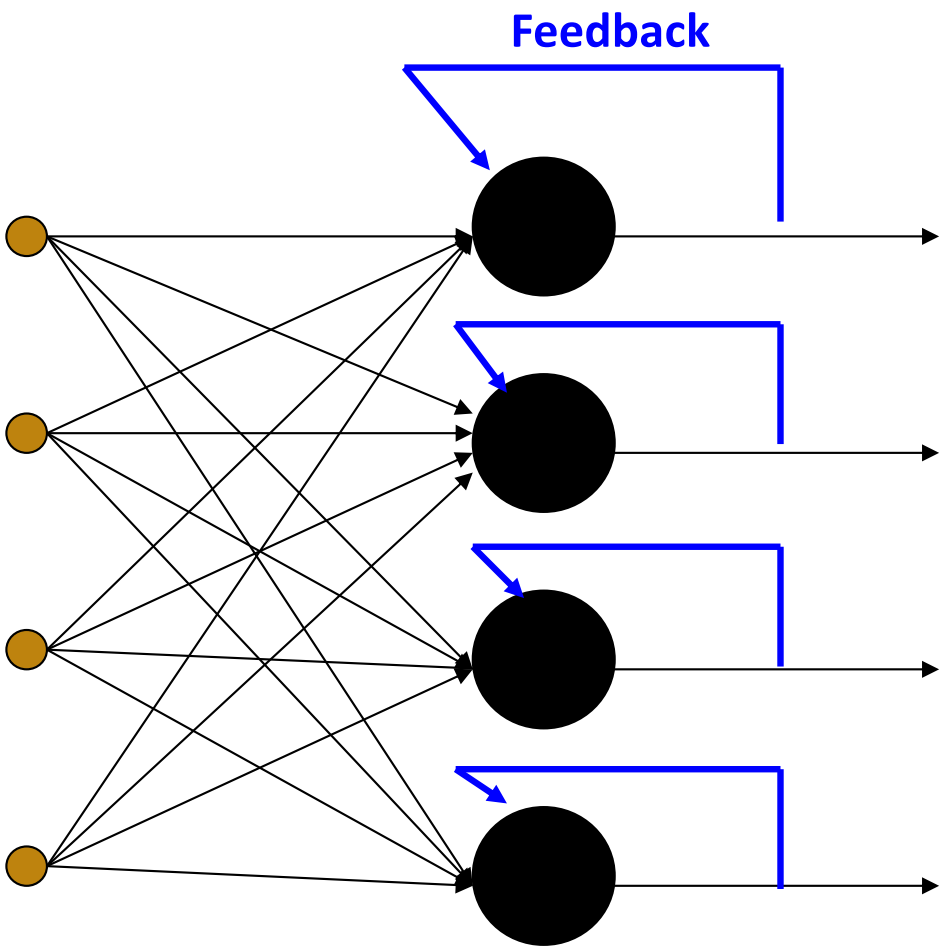




TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Feedforward vs Recurrent

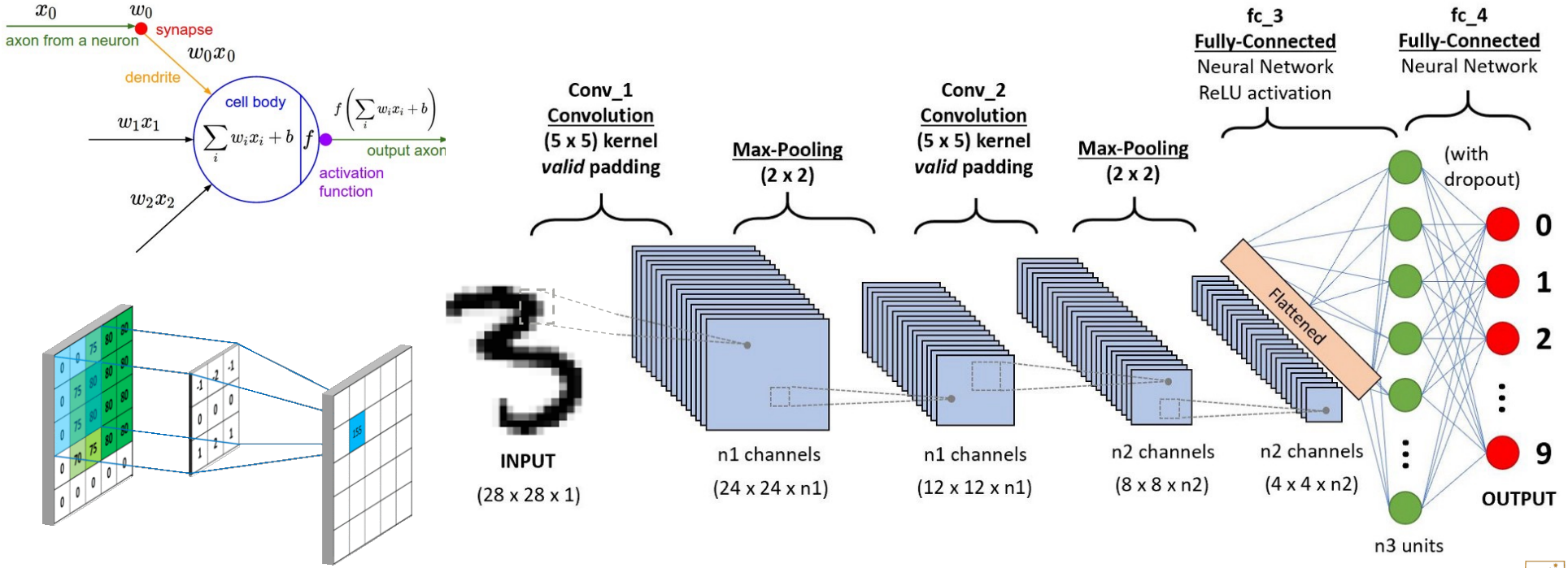
- FFN:
  - Information flows in one direction
  - Simple and efficient
  - Good for predicting a single output
- RNN:
  - Information flows back and forth
  - Learn temporal relationship in the input
  - Good for sequential data, e.g. LLM



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

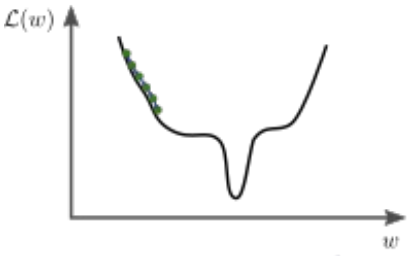
Dimension – Convolutional Neural Network

Essentially, f is a convolution kernel



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Neural Network Training



- Loss (L):

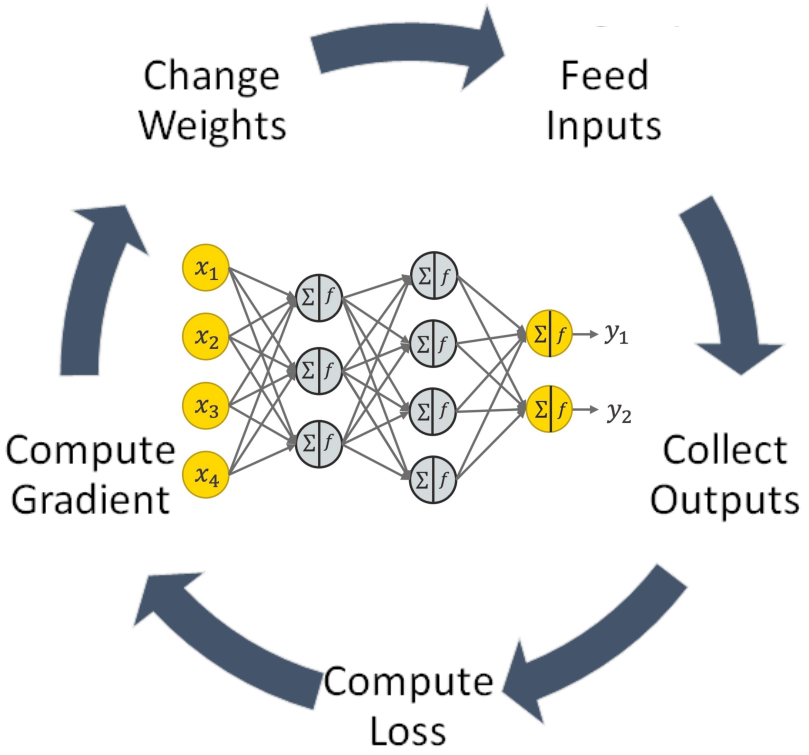
Cost of error, a measure of how well or poorly a model's predictions match the actual target values. The goal of training is to minimize this loss to make the predictions as accurate as possible.

- Backpropagation:

Backward propagation of errors, a fundamental algorithm used to train artificial neural network. It works by calculating the error between the network's predictions and the actual output. The error is then propagated back through the network, and the weights of the network are adjusted to reduce the error.

- Gradient descent (∇):

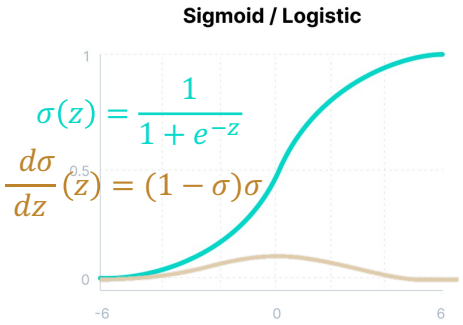
$w = w - \eta \nabla L(w)$, an optimization algorithm that is used to find the minimum of a function. It works by iteratively updating the parameters of a function until the function reaches a minimum. The gradient of a function is a vector that points in the direction of the steepest ascent of the function, updating the parameters of a function in the direction of the negative gradient.

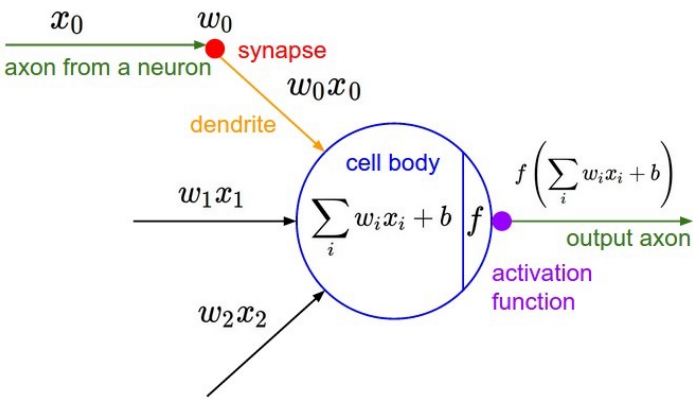


TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Example

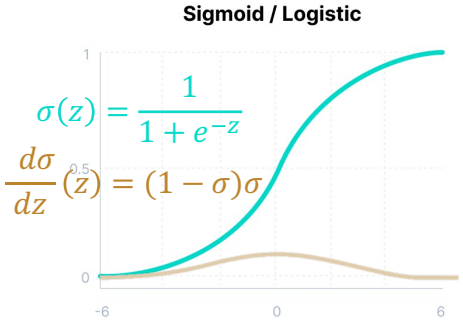
- Loss function is Mean Squared Error: $L =$
- Sigmoid activation function:
- A simple feedforward network with 2 layers and each layer has only 1 node



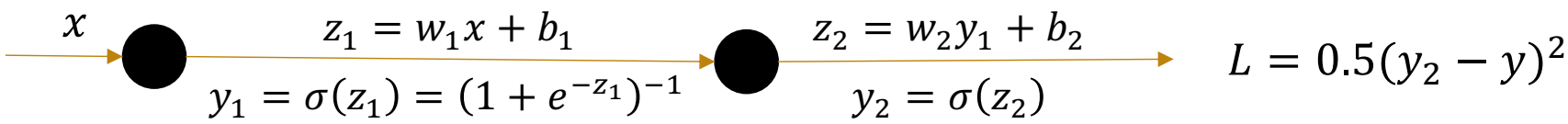


TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Example



- Loss function is Mean Squared Error: $L = 0.5(\hat{y} - y)^2$
- Sigmoid activation function: $\sigma(z) = \frac{1}{1+e^{-z}}$, $\frac{d\sigma}{dz}(z) = (1-\sigma)\sigma$
- A simple feedforward network with 2 layers and each layer has only 1 node



- Step 1 - Gradient of the output neuron: $\frac{\partial L}{\partial y_2} = y_2 - y$
- Step 2 - Gradient of the neuron in the output layer (chain rule!):

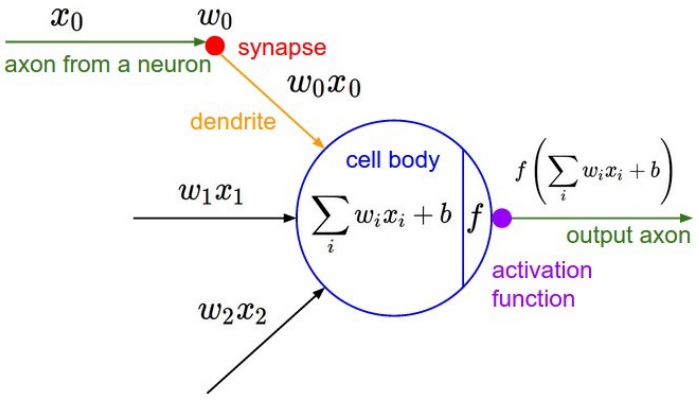
$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y_2} \cdot \frac{\partial y_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial w_2} = (y_2 - y) \cdot (1 - y_2) y_2 \cdot y_1$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial y_2} \cdot \frac{\partial y_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial b_2} = (y_2 - y) \cdot (1 - y_2) y_2$$

- Step 3 – Gradient of the neuron in the input layer (more chain rule!!!):

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y_2} \cdot \frac{\partial y_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial y_1} \cdot \frac{\partial y_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial w_1} = (y_2 - y) \cdot (1 - y_2) y_2 \cdot w_2 \cdot (1 - y_1) y_1 \cdot x$$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial y_2} \cdot \frac{\partial y_2}{\partial z_2} \cdot \frac{\partial z_2}{\partial y_1} \cdot \frac{\partial y_1}{\partial z_1} \cdot \frac{\partial z_1}{\partial b_1} = (y_2 - y) \cdot (1 - y_2) y_2 \cdot w_2 \cdot (1 - y_1) y_1$$

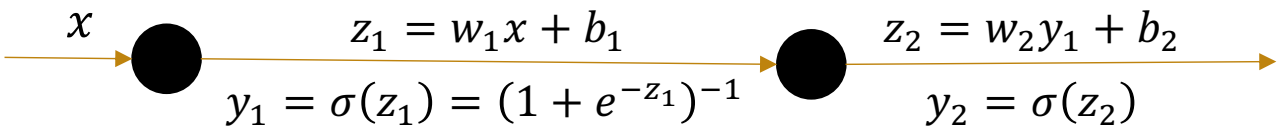


TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Training in action

- Let's assume some numbers
- $x = 1, y = 1, w_1 = 0.5, w_2 = 0.8, b_1 = 0, b_2 = 0, \eta = 0.1$

$$\nabla L(w_2) = (y_2 - y) \cdot (1 - y_2) y_2 \cdot y_1$$
$$\nabla L(b_2) = (y_2 - y) \cdot (1 - y_2) y_2$$
$$\nabla L(w_1) = (y_2 - y) \cdot (1 - y_2) y_2 \cdot w_2 \cdot (1 - y_1) y_1 \cdot x$$
$$\nabla L(b_1) = (y_2 - y) \cdot (1 - y_2) y_2 \cdot w_2 \cdot (1 - y_1) y_1$$
$$w = w - \eta \nabla L(w); \quad b = b - \eta \nabla L(b)$$



$$L = 0.5(y_2 - y)^2$$

step	w_1	b_1	z_1	y_1	$\nabla L(w/\mathbf{b}_1)$	w_2	b_2	z_2	y_2	$\nabla L(w_2)$	$\nabla L(b_2)$	L
1	0.5000	0.0000				0.8000	0.0000					
2												
3												
4												
5												

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Learning rate

$$w = w - \eta \nabla L(w)$$

The learning rate determines the size of the steps taken towards minimizing the loss function, influencing how quickly or slowly a model adjusts its weights and biases during training.



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Learning rate

$$w = w - \eta \nabla L(w)$$

The learning rate determines the size of the steps taken towards minimizing the loss function, influencing how quickly or slowly a model adjusts its weights and biases during training.

small learning rate leads to slow convergence

$\eta = 0.1$

step	w_1	b_1	z_1	y_1	$\nabla L(w/\mathbf{b}_1)$	w_2	b_2	z_2	y_2	$\nabla L(w_2)$	$\nabla L(b_2)$	L
1	0.5000	0.0000	0.5000	0.6225	-0.0168	0.8000	0.0000	0.4980	0.6220	-0.0554	-0.0889	0.0715
2	0.5017	0.0017	0.5034	0.6232	-0.0167	0.8055	0.0089	0.5151	0.6260	-0.0540	-0.0867	0.0696
3	0.5033	0.0034	0.5067	0.6240	-0.0165	0.8110	0.0176	0.5231	0.6278	-0.0542	-0.0869	0.0691
4	0.5049	0.0051	0.5100	0.6248	-0.0165	0.8164	0.0264	0.5331	0.6302	-0.0538	-0.0862	0.0682
5	0.5066	0.0067	0.5132	0.6256	-0.0164	0.8218	0.0350	0.5388	0.6318	-0.0535	-0.0859	0.0677



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Learning rate

$$w = w - \eta \nabla L(w)$$

The learning rate determines the size of the steps taken towards minimizing the loss function, influencing how quickly or slowly a model adjusts its weights and biases during training.

increasing learning rate can speed up training $\eta = 1$

step	w_1	b_1	z_1	y_1	$\nabla L(w/\mathbf{b}_1)$	w_2	b_2	z_2	y_2	$\nabla L(w_2)$	$\nabla L(b_2)$	L
1	0.5000	0.0000	0.5000	0.6225	-0.0168	0.8000	0.0000	0.4980	0.6220	-0.0554	-0.0889	0.0715
2	0.5168	0.0168	0.5336	0.6303	-0.0165	0.8555	0.0889	0.5566	0.6357	-0.0531	-0.0862	0.0648
3	0.5333	0.0333	0.5666	0.6379	-0.0163	0.9086	0.1751	0.6114	0.6491	-0.0508	-0.0830	0.0590
4	0.5497	0.0497	0.5993	0.6452	-0.0160	0.9595	0.2580	0.6622	0.6617	-0.0483	-0.0793	0.0539
5	0.5657	0.0657	0.6314	0.6522	-0.0157	1.0080	0.3373	0.7095	0.6732	-0.0457	-0.0751	0.0493



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Learning rate

$$w = w - \eta \nabla L(w)$$

The learning rate determines the size of the steps taken towards minimizing the loss function, influencing how quickly or slowly a model adjusts its weights and biases during training.

increasing learning rate can speed up training **BUT!!!** $\eta = 10$

step	w_1	b_1	z_1	y_1	$\nabla L(w/\mathbf{b}_1)$	w_2	b_2	z_2	y_2	$\nabla L(w_2)$	$\nabla L(b_2)$	L
1	0.5000	0.0000	0.5000	0.6225	-0.0168	0.8000	0.0000	0.4980	0.6220	-0.0554	-0.0889	0.0715
2	0.6680	0.1680	0.8359	0.6976	-0.0126	1.3541	0.8890	1.0191	0.7348	-0.0407	-0.0661	0.0351
3	0.7941	0.2939	1.0880	0.7479	-0.0092	1.7612	1.5500	1.4579	0.8112	-0.0298	-0.0484	0.0177
4	0.8864	0.3862	1.2726	0.7811	-0.0067	2.0596	2.0344	1.8551	0.8647	-0.0219	-0.0356	0.0089
5	0.9533	0.4532	1.4065	0.8012	-0.0049	2.2777	2.3903	2.2228	0.9023	-0.0160	-0.0260	0.0044



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

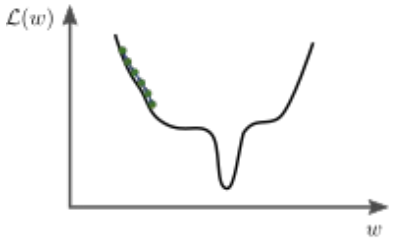
Learning rate

$$w = w - \eta \nabla L(w)$$

The learning rate determines the size of the steps taken towards minimizing the loss function, influencing how quickly or slowly a model adjusts its weights and biases during training.

increasing learning rate can speed up training

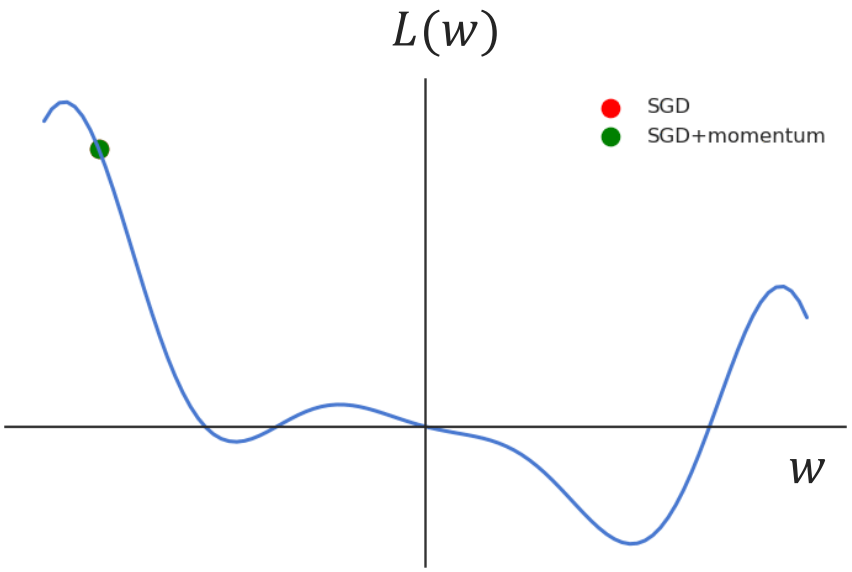
BUT!!! large η is not always good



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Optimizer

mathematical tools/algorithms to adjust parameters



$$\Delta w^t = \eta \nabla L(w) + \gamma \Delta w^{t-1}$$

- **Stochastic Gradient Descent (SGD)**

updates model parameters using a randomly selected subset of training data (a mini-batch) at each iteration. This helps speed up training and allows for updates more frequently compared to traditional gradient descent, which uses the entire dataset.

- **Adaptive Gradient Algorithm (Adagrad)**

adapts the learning rate for each parameter based on the historical gradients. It gives more weight to infrequently updated parameters and less to frequently updated ones, making it suitable for sparse data.

- **Root Mean Square Propagation (RMSprop)**

modifies Adagrad by using a moving average of the squared gradients to normalize the learning rate. This helps to avoid the rapid decay of learning rates that can occur with Adagrad, making it more effective for non-stationary objectives.

- **Adaptive Moment Estimation (Adam)**

improves upon SGD and RMSprop by adding a fraction of the previous update to the current update. It maintains a moving average of both the gradients and their squared values, allowing it to adaptively adjust the learning rate for each parameter.

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Loss function

Cost of error, a measure of how well or poorly a model's predictions match the actual target values. The goal of training is to minimize this loss to make the predictions as accurate as possible.

Training (70%) + Validation (20%) + Test set (10%)

Things the network tries to fit

Things the network uses to exam itself

Things the network has never seen and you use to exam it

- Regression:
  - Mean Squared Error Loss (MSE):

$$L(y,\hat{y}) = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

  - Mean Absolute Error Loss (MAE):

$$L(y,\hat{y}) = \frac{1}{N}\sum_{i=1}^{N}|y_i - \hat{y}_i|$$

  - Mean Squared Logarithmic Error Loss (MSLE):

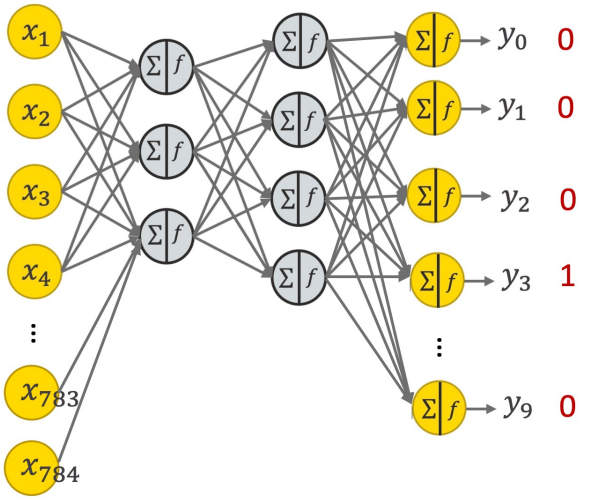
$$L(y,\hat{y}) = \frac{1}{N}\sum_{i=1}^{N}[\log(y_i+1) - \log(\hat{y}_i+1)] = \frac{1}{N}\sum_{i=1}^{N}\left(\log\frac{y_i+1}{\hat{y}_i+1}\right)$$

- Classification:
  - Cross-entropy Loss:

$$L(y,\hat{y}) = -\sum_{i=1}^{N}[y_i \cdot \log(\hat{y}_i)]$$

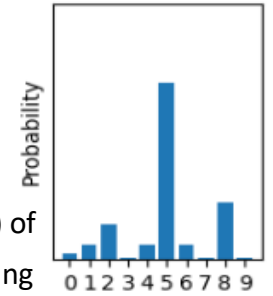
  - Binary Cross Entropy Loss:

$$L(\hat{y}) = -\frac{1}{N}\sum_{i=1}^{N}\{y_i \cdot \log[p(y_i)] + (1-y_i)\cdot\log[1-p(y_i)]\}$$



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Cross-entropy and softmax



For tasks like recognizing handwritten digit, we can use softmax activation function in the output layer to predicts the probability ($\hat{y}$) of the image being 0, 1, …, 9, i.e.,$\hat{y}_i = \sigma(z_i) = \frac{e^{z_i}}{\sum_{i=0}^{9} e^{z_i}}$. The softmax function naturally gives $\Sigma\sigma(z_i) = 1$ with each elements representing the probability of the output. In other word, the probability of the input being 0, 1, …, 9.



Cross entropy, $L(y, \hat{y}) = -\sum_{i=1}^{N}[y_i \cdot \log(\hat{y}_i)]$, measures the difference between two probability distributions (i.e. the true vs predicted). We need $\hat{y}$ to be as close to 1 as possible for only the label where y=1, and as close to 0 as possible for all other labels.

Expand the cross-entropy loss function

$$L(y, \hat{y}) = -\sum_{k=0}^{9}[y_k \cdot \log(\hat{y}_k)] = -\sum_{k=0}^{9}\left[y_k \cdot \log\left(\frac{e^{z_k}}{\sum_{j=0}^{9} e^{z_j}}\right)\right] = -\sum_{k=0}^{9} y_k \left[z_k - \log\left(\sum_{j=0}^{9} e^{z_j}\right)\right]$$

Calculate the derivative over the z_j

$$\frac{\partial L}{\partial z_i} = -\sum_{k=0}^{9} y_k \left[\frac{\partial z_k}{\partial z_i} - \frac{\partial}{\partial z_i}\log\left(\sum_{j=0}^{9} e^{z_j}\right)\right] = -\sum_{k=0}^{9} y_k \delta_{ki} + \sum_{k=0}^{9} y_k \frac{e^{z_i}}{\sum_{j=0}^{9} e^{z_j}} = -y_i + \frac{e^{z_i}}{\sum_{j=0}^{9} e^{z_j}} = -y_i + \hat{y}_i$$

For the correct label (y_i=1), if the network predicts a higher probability ($\hat{y}_i > y_i$), the gradient is positive, meaning that z_i and hence $\hat{y}_i$ should be decreased to reduce the loss. When $\hat{y}_i < y_i$, the gradient is negative, z_i and $\hat{y}_i$ should be increased to reduce the loss.
For incorrect labels (y_i=0), the gradient is simply $\hat{y}_i$, encouraging the network to decrease the probability of incorrect labels.

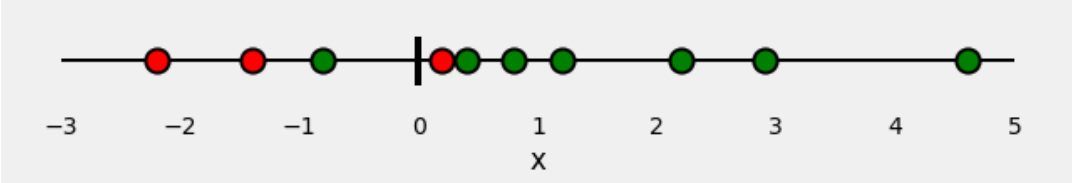


TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

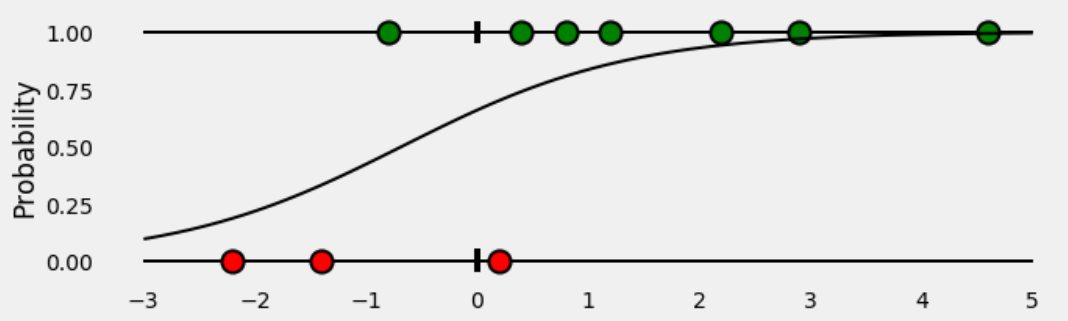
Binary Cross Entropy

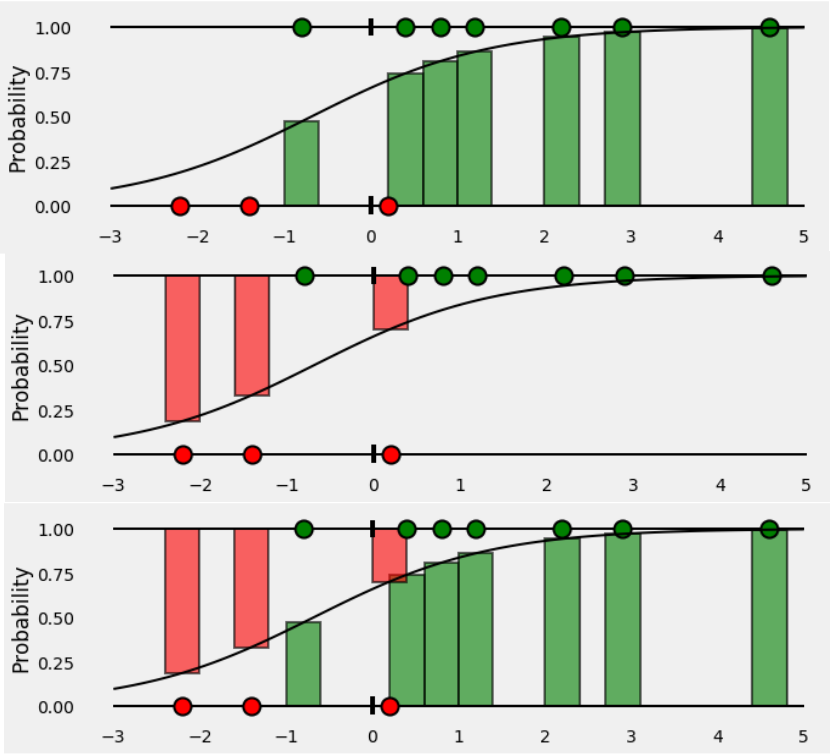
$$L = -\frac{1}{N}\sum_{i=1}^{N}\{y_i \cdot \log[p(y_i)] + (1-y_i) \cdot \log[1-p(y_i)]\}$$

Task: classify the red and green points given x



ML language -> what is the probability of the point being green?
i.e., P(green is green) = 1 and P(green is red)=0
Labelling green as y=1 and red as y=0, then p(y) is our model, representing the probability of the point being green





Credit: Daniel Godoy

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Australia's Artificial Intelligence Ethics Principles

Australia's 8 Artificial Intelligence (AI) Ethics Principles are designed to ensure AI is safe, secure and reliable.

- Department of Industry, Science and Resources (November 2019)

- **Human, societal and environmental wellbeing**: AI systems should benefit individuals, society and the environment
- **Human-centred values**: AI systems should respect human rights, diversity, and the autonomy of individuals
- **Fairness**: AI systems should be inclusive and accessible, and should not involve or result in unfair discrimination against individuals, communities or groups
- **Privacy protection and security**: AI systems should respect and uphold privacy rights and data protection, and ensure the security of data
- **Reliability and safety**: AI systems should reliably operate in accordance with their intended purpose
- **Transparency and explainability**: There should be transparency and responsible disclosure so people can understand when they are being significantly impacted by AI, and can find out when an AI system is engaging with them
- **Contestability**: When an AI system significantly impacts a person, community, group or environment, there should be a timely process to allow people to challenge the use or outcomes of the AI system.
- **Accountability**: People responsible for the different phases of the AI system lifecycle should be identifiable and accountable for the outcomes of the AI systems, and human oversight of AI systems should be enabled



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Australia's Guidance for AI adoption

The Guidance for AI Adoption sets out 6 practices for responsible AI adoption across Australia's economy.
– National Artificial Intelligence Centre, Department of Industry, Science and Resources (October 2025)

1 **Decide who is accountable**

Establish end-to-end accountability and robust AI governance.

2 **Understand impacts and plan accordingly**

Ensure stakeholder rights and fair treatment before and throughout adoption.

3 **Measure and manage risks**

Implement AI-specific risk management proportionate to the system and its potential harms.

4 **Share essential information**

Provide appropriate transparency and explainability to users and others in the AI supply chain.

5 **Test and monitor**

Evaluate AI systems and monitor their quality, reliability and protection after deployment.

6 **Maintain human control**

Integrate meaningful human oversight and retain the ability to intervene when needed.

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Australia's National AI Plan

The National AI Plan sets the whole-of-economy direction for AI development, adoption and governance.
– Department of Industry, Science and Resources (December 2025)

1 Capture the opportunities

- Build smart infrastructure
- Back Australian AI capability
- Attract investment

2 Spread the benefits

- Scale responsible AI adoption
- Support and train Australians
- Improve public services

3 Keep Australians safe

- Mitigate AI-related harms
- Promote responsible practices
- Partner on global norms

Regulatory direction

Existing Australian laws remain the foundation for managing AI risks. Government agencies and regulators retain responsibility in their sectors, with targeted legal changes introduced where gaps or specific harms emerge.

AI Safety Institute

The Plan establishes an institute to monitor, test and share evidence about emerging AI capabilities, risks and harms.

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Government Use of AI

The APS framework combines increased AI adoption with mandatory agency-level accountability and risk controls.
– Department of Finance and Digital Transformation Agency (2024–2025)

The policy framework

1 National AI Assurance Framework

21 June 2024. A nationally consistent approach for Australian, state and territory governments.

2 AI Plan for the APS

12 November 2025. Expands safe AI use to improve services, policy outcomes and productivity.

3 Responsible Use Policy v2.0

Effective 15 December 2025. Mandatory for non-corporate Commonwealth entities.

Mandatory under Policy

Transparency	Publish and annually update an AI transparency statement.
Accountability	Designate accountable officials and use-case owners.
Use-case register	Record in-scope uses and share the register with the DTA.
Impact assessment	Assess impacts during design and before deployment.
Monitoring	Review deployed systems and reassess material changes.
Staff capability	Train all staff in responsible and ethical AI use.

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Internationally

The Blueprint for an AI Bill of Rights (2022) from the White House outlines key principles for the responsible use of artificial intelligence. It emphasizes the need for AI systems to be designed and used in ways that respect individual rights and promote fairness. The document focuses on five main areas, including **Safety and Security, Non-Discrimination**, **Privacy, Notice and Explanation, Human Alternatives.** Current federal strategy is **America's AI Action Plan (2025)**, emphasizing innovation, infrastructure, national security and international leadership. State regulation is also increasingly important.

The primary legislative framework for regulating AI in the EU is the EU AI Act. It was published in the EU Official Journal on July 12, 2024, and is the first comprehensive horizontal legal framework for the regulation of AI across the EU. The EU AI Act enters into force on August 1, 2024, and will be effective from August 2, 2026, except for the specific provisions listed in Article 113. The key features of the EU AI Act include **Risk-Based Classification, Safety and Accountability, Prohibitions, Compliance and Enforcement.**

Canada: The Directive on Automated Decision-Making (2019, updated 2025) governs federal public-sector. New 2026 proposals include privacy reform (**Bill C-36**) and the proposed **Safe Social Media Act (Bill C-34)**

China: Multiple binding regulations: Algorithmic Recommendation Provisions (2021); Deep Synthesis (Deepfake) Provisions (2022); Interim Measures for Generative AI (2023), AI content labelling rules (2025), Companion-style AI interaction services (2026).

India: The **India AI Governance Guidelines (2026)** establish a principle-based, "techno-legal" approach built on existing IT, data-protection and sectoral laws. India has also established the **AI Governance and Economic Group (AIGEG)** and an **AI Safety Institute**

Japan: first national **AI Act** in 2025, the *Act on the Promotion of Research and Development and Utilization of AI-related Technology. A* national promotion / governance framework and complements the **AI Guidelines for Business.**

South Korea: The **AI Basic Act** entered into force on **22 Jan. 2026**, making South Korea another major jurisdiction with comprehensive AI-specific legislation. It establishes national AI governance and includes **transparency and safety obligations**, including provisions for **high-impact AI**.

Singapore: Continues a predominantly **soft-law and assurance-based approach**: Model AI Governance Framework, **Generative AI framework (2024)**, AI Verify and related testing tools. In **Jan. 2026**, Singapore launched a world-first **Model AI Governance Framework for Agentic AI**, updated in May 2026, emphasizing bounded autonomy, human accountability and lifecycle controls.

UK: primarily governed through existing sector-specific and cross-cutting laws, supported by the **AI Opportunities Action Plan (2025)** and the AI Security Institute. The UK has increasingly adopted **targeted AI legislation**, including 2026 measures addressing illegal AI-generated content.

(incomplete list…)

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Neural network example – prepare dataset

```
import tensorflow.compat.v1 as tf
from tensorflow.keras import layers
from tensorflow.keras.optimizers import Adam

# load training data (X_train, Y_train)
# load validation data (X_valid, Y_valid)
# covert them into tensorflow format
x_train = tf.data.Dataset.from_tensor_slices(X_train); x_val = tf.data.Dataset.from_tensor_slices(X_valid)
y_train = tf.data.Dataset.from_tensor_slices(Y_train); y_val = tf.data.Dataset.from_tensor_slices(Y_valid)

# This is the size of a single training batch (training is not done on the entire test set at once, but on batches of data)
batch_size = 64

# package the data
training_data = tf.data.Dataset.zip((x_train, y_train)).shuffle(X_train.shape[0]).batch(batch_size)
validation_data = tf.data.Dataset.zip((x_val, y_val)).shuffle(X_valid.shape[0]).batch(batch_size)

# fancy features
callbacks = [tf.keras.callbacks.ReduceLROnPlateau(patience = 20), # If loss does not improve for 20 epochs, reduce learning rate
   tf.keras.callbacks.EarlyStopping(patience = 50)] # If loss does not improve for 50 epochs, stop the learning
```



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Neural network example – define network structure

```
# Input size = number of params
input_layer = layers.Input(shape = X_train.shape[-1])

# Hidden fully-connected layers
output = layers.Dense(64)(input_layer)
# Batch normalization = normalize the training weights at every batch.
output = layers.BatchNormalization()(output)
output = layers.ReLU()(output)
output = layers.Dense(128)(output)
output = layers.BatchNormalization()(output)
output = layers.ReLU()(output)
output = layers.Dense(64)(output)
output = layers.BatchNormalization()(output)
output = layers.ReLU()(output)
# Last layer output shape
output = layers.Dense(Y_train.shape[-1])(output)

model = tf.keras.Model(inputs = [input_layer], outputs = [output])
model.summary()
model.compile(loss = 'mse', optimizer = Adam(learning_rate = 1e-3))
```

```
Layer (type)                 Output Shape              Param #
=================================================================
input_1 (InputLayer)         [(None, 9)]               0
_________________________________________________________________
dense (Dense)                (None, 64)                640
_________________________________________________________________
batch_normalization (BatchNo (None, 64)                256
_________________________________________________________________
re_lu (ReLU)                 (None, 64)                0
_________________________________________________________________
dense_1 (Dense)              (None, 128)               8320
_________________________________________________________________
batch_normalization_1 (Batch (None, 128)               512
_________________________________________________________________
re_lu_1 (ReLU)               (None, 128)               0
_________________________________________________________________
dense_2 (Dense)              (None, 64)                8256
_________________________________________________________________
batch_normalization_2 (Batch (None, 64)                256
_________________________________________________________________
re_lu_2 (ReLU)               (None, 64)                0
_________________________________________________________________
dense_3 (Dense)              (None, 54)                3510
=================================================================
Total params: 21,750
Trainable params: 21,238
Non-trainable params: 512
_________________________________________________________________
```



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

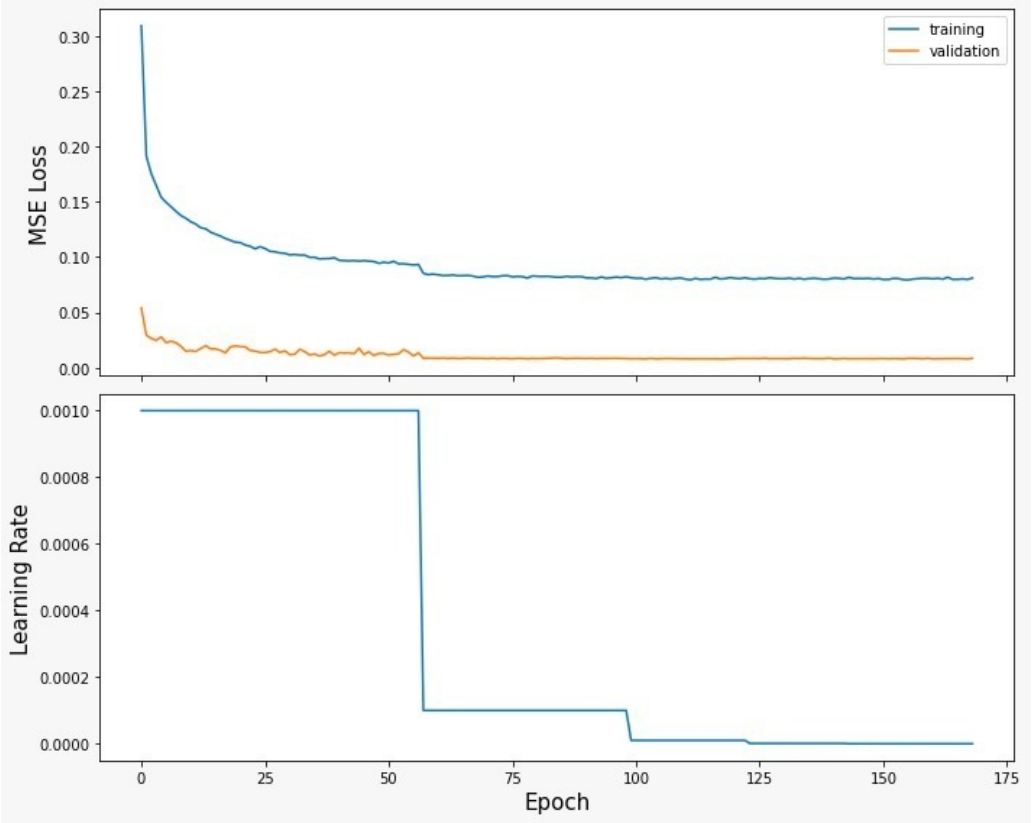
Neural network example – train network

```
history = model.fit(training_data,
                    epochs = 300,
                    batch_size = batch_size,
                    validation_data = validation_data,
                    callbacks = callbacks,
                    shuffle = True # shuffle the data in each batch
                    )

# get histories of loss for training set
(history.history['loss']) and validation set
(history.history['val_loss']) as well as the learning rate
(history.history['lr'])

# make prediction by calling model.predict(X_test)

# save the network by calling model.save('emulator')
```



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

prerequisite

We will start at 1:05pm, before that, please

git clone https://github.com/qyx268/astr4004-8004-2026.git

To open a gz file, do gzip -d file.gz

install tensorflow, matplotlib, numpy



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C

Assignment 4 (due on Oct 26)

Design a neural network for regression *(bonus point if the model is astrophysical)*

- with at least 5 neurons;
- involving either relu (max(0,x)) or sigmoid (1/(exp(-x)+1)) activation functions;
- having a loss that within 10 steps
  - monotonically decreases when the learning rate is 0.1,
  - but can increase within 10 steps when the learning rate is >0.1.

The report should include

- a description of the model or regression problem you are solving; (10%)
- a graph illustrating the network structure; (10%)
- equations of gradient for all neurons; (30%)
- two tables listing weights, gradients, outputs and loss for 10 steps
  - learning rate = 0.1, (25%)
  - learning rate > 0.1 (25%)

Bonus point might maximize the score for this assignment and this assignment only



TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY) CRICOS PROVIDER CODE: 00120C